

CV9: Trvalé uloženie údajov pomocou Room - Zadanie

1 Zadanie cvičenia

Vytvorte jednoduchú Android aplikáciu v jazyku Kotlin s použitím Jetpack Compose a knižnice Room. Aplikácia bude predstavovať osobný *Movie Watchlist*, teda zoznam filmov, ktoré si používateľ chce pozrieť alebo už videl. Cieľom cvičenia je, aby ste si precvičili trvalé uloženie údajov do lokálnej databázy a zároveň nadviazali na prácu s `ViewModel` triedou z predchádzajúceho cvičenia.

Balík aplikácie použite v tvare `com.example.cv9`.

2 Požadované správanie aplikácie

Po spustení aplikácie sa má zobrazíť hlavná obrazovka so sekciou na pridanie filmu a so zoznamom už uložených filmov. Ak v databáze ešte nie sú žiadne údaje, aplikácia má zobrazíť prázdny stav s jednoduchou informačnou správou.

Používateľ musí mať možnosť pridať nový film zadaním názvu. Nový film sa po uložení zapíše do Room databázy a zobrazí sa v zozname. Každý novo pridaný film sa má považovať za nevidený. Pri pridaní sa zároveň uloží čas vytvorenia záznamu.

Zoznam filmov sa musí zobrazovať pomocou `LazyColumn`. Každá položka má obsahovať minimálne názov filmu a čas pridania. Ak film ešte nebol označený ako videný, položka má zobrazovať jednoduchú informáciu o tom, že film ešte nebol pozretý.

Používateľ musí vedieť označiť film ako videný. Pri tejto akcii sa má zobrazíť jednoduchý dialóg, v ktorom používateľ vyberie hodnotenie v rozsahu 1 až 5 a doplní krátky komentár, teda dôvod svojho hodnotenia. Po potvrdení sa film uloží ako videný, uloží sa k nemu komentár, hodnotenie a čas označenia za videný.

Ak je film označený ako videný, v zozname sa má zobrazovať jeho hodnotenie, čas pozretia a skrátený jednoriadkový náhľad komentára. Ak je komentár dlhší, text sa má skrátiť a ukončiť tromi bodkami.

Používateľ musí vedieť film opäť označiť ako nevidený. V takom prípade sa z filmu odstráni informácia o tom, že bol videný, vrátane hodnotenia, komentára a času pozretia.

Používateľ musí vedieť film odstrániť zo zoznamu.

3 Filter a triedenie

Aplikácia musí obsahovať jednoduchý filter, pomocou ktorého bude možné zobrazit' všetky filmy, iba videné filmy alebo iba nevidené filmy.

Okrem filtra musí aplikácia obsahovať aj jednoduché triedenie. Predvolené triedenie má byť podľa času pridania od najnovšieho filmu. Druhá možnosť triedenia má byť podľa času označenia za videný, pričom videné filmy sa zobrazia pred nevidenými filmami.

4 Požiadavky na rozloženie kódu

Používateľské rozhranie vytvorte v Jetpack Compose. Stav obrazovky a obsluha udalostí majú byť riešené cez `ViewModel`. Údaje nesmú byť držané iba v `remember` alebo v obyčajnom zozname v composable funkcii. Trvalé uloženie musí byť realizované cez Room databázu.

V projekte vytvorte minimálne entitu filmu, DAO rozhranie a databázovú triedu. Odporúčanou súčasťou riešenia je aj jednoduchý repozitár, ktorý bude sprostredkovať komunikáciu medzi `ViewModel` triedou a DAO vrstvou.

Vhodná štruktúra súborov môže vyzerat' napríklad takto:

```
MainActivity.kt
MovieEntity.kt
MovieDao.kt
MovieDatabase.kt
MovieRepository.kt
MovieUiState.kt
MovieWatchlistViewModel.kt
MovieWatchlistScreen.kt
```

5 Odporúčaný dátový model

Každý film by mal obsahovať identifikátor, názov filmu, informáciu o tom, či bol videný, hodnotenie, komentár, čas pridania a čas označenia za videný. Odporúčaný model môže vyzerat' napríklad takto:

```
@Entity(tableName = "movies")
data class MovieEntity(
    @PrimaryKey(autoGenerate = true) val id: Long = 0,
    val title: String,
    val isWatched: Boolean = false,
    val rating: Int? = null,
    val comment: String = "",
    val createdAt: Long,
    val watchedAt: Long? = null
)
```

Na uchovanie času použijete číselné hodnoty typu `Long`. Vďaka tomu sa budete môcť jednoducho rozhodovať pri triedení a zároveň si vyhnete potrebu typových konvertorov.

6 Odporúčanie pre používateľské rozhranie

Hornú časť obrazovky môžete využiť na krátku hlavičku a formulár na pridanie názvu filmu. Pod touto časťou je vhodné umiestniť filter a triedenie. Hlavnú plochu obrazovky by mal zaberáť zoznam filmov.

Jedna položka zoznamu môže byť navrhnutá ako karta. Pri nevidenom filme môže karta obsahovať názov, čas pridania a tlačidlo na označenie za videný. Pri videnom filme môže karta obsahovať názov, hodnotenie, čas pozretia, náhľad komentára a akcie na úpravu, zmenu stavu alebo odstránenie.

7 Ukážka DAO rozhrania

Nasledujúca ukážka predstavuje minimálny rozsah databázových operácií, ktoré budete v tomto cvičení potrebovať.

```
@Dao
interface MovieDao {

    @Query("SELECT * FROM movies ORDER BY createdAt DESC")
    suspend fun getAllMovies(): List<MovieEntity>

    @Insert
    suspend fun insertMovie(movie: MovieEntity)

    @Update
    suspend fun updateMovie(movie: MovieEntity)

    @Delete
    suspend fun deleteMovie(movie: MovieEntity)
}
```

8 Ukážka logiky vo ViewModel triede

Vo `ViewModel` triede spracujte používateľské udalosti tak, aby composable funkcie neobsahovali databázovú logiku. Jedna z dôležitých funkcií bude napríklad pridanie filmu.

```
fun addMovie() {
    val trimmedTitle = uiState.titleInput.trim()
    if (trimmedTitle.isBlank()) {
        uiState = uiState.copy(
            errorMessage = "Please enter a movie title."
        )
        return
    }
}
```

```

viewModelScope.launch {
    repository.insertMovie(
        MovieEntity(
            title = trimmedTitle,
            createdAt = System.currentTimeMillis()
        )
    )
    loadMovies()
}
}

```

Podobným spôsobom spracujte aj uloženie komentára a hodnotenia, zmenu filmu na nevidený a odstránenie položky.

9 Technická poznámka k Room a KSP konfigurácii

Pri použití knižnice Room v Kotlin projekte je potrebné doplniť nielen Room závislosti, ale aj KSP konfiguráciu. Bez nej sa nevygeneruje databázová implementácia a aplikácia databázu nedokáže vytvoriť.

Ak projekt používa version catalogs, doplňte do súboru `libs.versions.toml` verziu a plugin alias pre KSP.

```

[versions]
ksp = "2.0.21-1.0.26"

[plugins]
ksp = { id = "com.google.devtools.ksp", version.ref = "ksp" }

```

Plugin je potom potrebné deklarovať v koreňovom súbore `build.gradle.kts` a aktivovať v module `app`.

```

plugins {
    alias(libs.plugins.android.application) apply false
    alias(libs.plugins.kotlin.android) apply false
    alias(libs.plugins.kotlin.compose) apply false
    alias(libs.plugins.ksp) apply false
}

```

```

plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.kotlin.android)
    alias(libs.plugins.kotlin.compose)
    alias(libs.plugins.ksp)
}

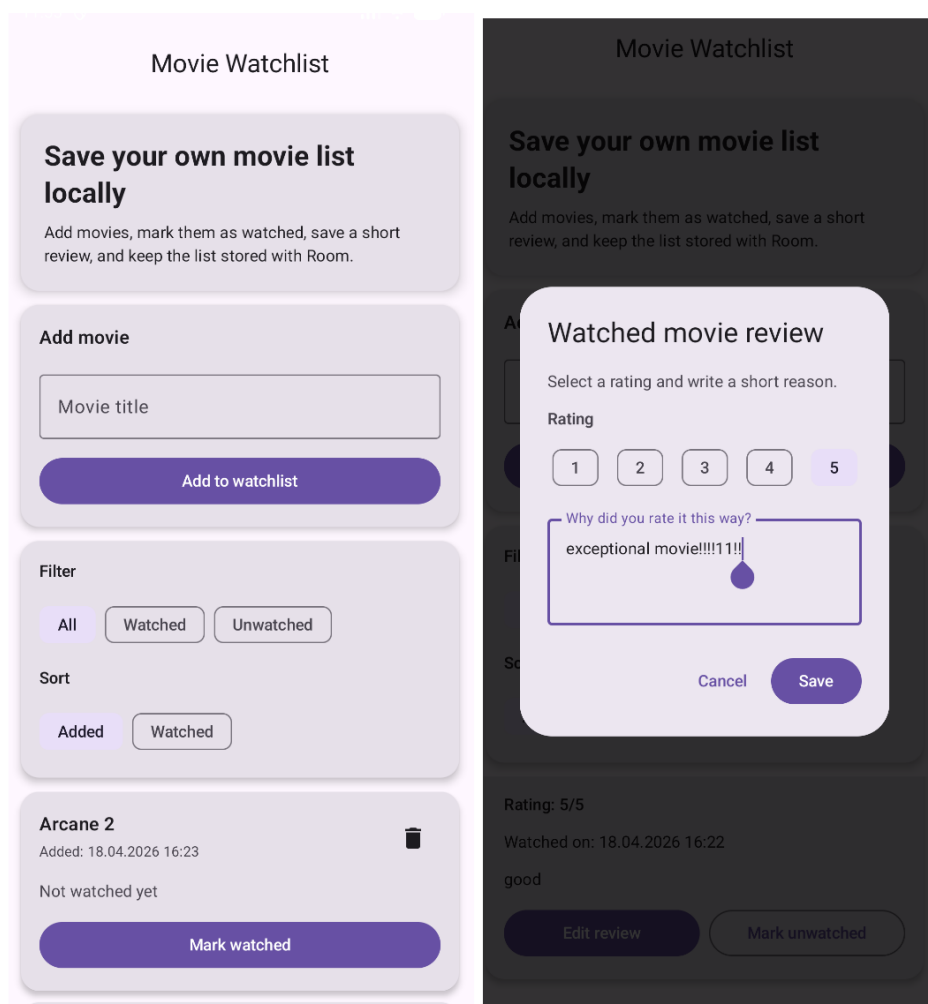
```

Do závislostí v module následne doplňte Room runtime, Kotlin rozšírenia a Room compiler cez KSP.

```
dependencies {  
    implementation("androidx.room:room-runtime:2.8.4")  
    implementation("androidx.room:room-ktx:2.8.4")  
    ksp("androidx.room:room-compiler:2.8.4")  
  
    implementation("androidx.lifecycle:lifecycle-viewmodel -  
        compose:2.8.6")  
    implementation("androidx.lifecycle:lifecycle-runtime-compose  
        :2.8.6")  
}
```

10 Výsledok cvičenia

Po dokončení budete mať jednooprazovkovú aplikáciu, ktorá pracuje s databázou Room a uchováva zoznam filmov aj po zatvorení aplikácie. Precvičíte si tým prepojenie Compose, ViewModel triedy a lokálnej databázy. Zároveň si upevníte rozdiel medzi dočasným stavom obrazovky a trvalými údajmi aplikácie.



Obr. 1: Ukážka výsledného zoznamu filmov